Field 1: Urban Taxi & Ride-Sharing Systems

1. Spatio-Temporal Autonomous Taxi Dispatching (MOD)

- **The Goal:** Optimize fleet utility and minimize passenger wait times using spatial indexing.
- **Detailed Tasks:**
    - **Index Design:** Implement an R-Tree index in SQLite to store moving taxi coordinates (x, y) updated via TraCI.
    - **Request Handling:** Generate stochastic passenger trip requests; use a K-Nearest Neighbor (KNN) query to find the closest idle taxis.
    - **Dispatch Logic:** Write a Python controller that assigns the taxi in the DB and uses TraCI's setRoute to navigate to the passenger.
    - **Availability Management:** Use atomic transactions to update taxi status from IDLE to PICKUP to OCCUPIED.
    - **Rebalancing:** Periodically query the DB for areas with high request density but low taxi supply and "rebalance" empty taxis to those zones.
- **Performance Evaluation Scale:**
    - **Network:** 3 km × 3 km urban grid (>500 edges).
    - **Scale:** 2,000 background vehicles; 300 active taxis; 1,500 passenger requests.
    - **Time:** 3,600s (1 hour) peak demand.
    - **Topic Metrics: Average Passenger Wait Time (AWT)** and **Taxi Empty-Mile Ratio**. Analyze how the spatial index depth correlates with dispatch success rates as fleet density triples.

2. Transactional Integrity in Shared-Ride Matching

- **The Goal:** Maximize vehicle occupancy through complex relational constraints without overbooking.
- **Detailed Tasks:**
    - **Relational Schema:** Design a schema for Vehicles, Current_Passengers, and Trip_Requests with seat-count constraints.
    - **Overlap Matching:** Write a SQL query that identifies passengers with overlapping origin-destination paths within a specific time window Δ t.
    - **Capacity Enforcement:** Use SQL CHECK constraints and transactions to ensure a vehicle never accepts a passenger if current_load + new_request > capacity.
    - **Dynamic Rerouting:** Use TraCI to add intermediate stops to a vehicle's route whenever a new shared-ride match is confirmed in the DB.
    - **Preference Filtering:** Include user preferences (e.g., "maximum 1 extra stop") in the SQL matching query.
- **Performance Evaluation Scale:**
    - **Network:** 5km × 5km mixed-use area.
    - **Scale:** 500 shared-ride vehicles; 3,000 trip requests; 4 seats per vehicle.
    - **Time:** 7,200s (2-hour window).
    - **Topic Metrics: Average Vehicle Occupancy (AVO)** and **Total Vehicle Miles**

Traveled (VMT) Reduction. Evaluate the "Rejection Rate" caused by transaction timeouts during peak matching bursts.

3. Dynamic Surge Pricing and Demand-Supply Equilibrium

- **The Goal:** Use analytical window functions to balance urban transport demand via economic incentives.
- **Detailed Tasks:**
    - **Zonal Aggregation:** Divide the SUMO map into districts; use SQL window functions to calculate demand density in the last T minutes.
    - **Price Logic:** Implement a Python-based pricing engine that updates a Price_Index table based on the demand/supply ratio of each district.
    - **Agent Behavior:** Assign a "Willingness-to-Pay" attribute to SUMO vehicles; vehicles query the DB and choose to "cancel" or "divert" if prices are too high.
    - **Revenue Tracking:** Log all completed transactions to evaluate platform profitability.
    - **Elasticity Analysis:** Use the DB to store historical price-vs-demand data to calibrate the pricing model.
- **Performance Evaluation Scale:**
    - **Network:** 10 distinct districts in a city center.
    - **Scale:** 5,000 agents with varying price sensitivities.
    - **Time:** 14,400s (4-hour morning rush).
    - **Topic Metrics: Congestion Smoothing Factor** (variance in average speed across districts) and **Request Fulfillment Rate**. Measure price elasticity by correlating DB-logged prices with agent "Divergence" rates.

Field 2: Vehicle Navigation & Infrastructure

4. Congestion-Aware Navigation with Persistent Edge-Weights

- **The Goal:** Improve city-wide travel times by using the DB as a dynamic graph-weight repository.
- **Detailed Tasks:**
    - **Graph Schema:** Represent the road network in SQLite (Edges, Nodes, Weights).
    - **Live Updates:** Use TraCI to measure real-time speeds; update the Weights table every N seconds.
    - **External Pathfinding:** Use Python to run a Dijkstra/A* algorithm that fetches weights from SQLite rather than using SUMO's internal static routing.
    - **Route Injection:** Force "Smart Vehicles" onto these DB-calculated paths via TraCI.
    - **Incident Management:** Manually close an edge in the DB (simulating an accident) and observe how the query-driven navigation reroutes traffic.
- **Performance Evaluation Scale:**
    - **Network:** City-scale network (>10,000 edges).
    - **Scale:** 5,000 "Smart Vehicles" fetching weights from DB; 2,000 "Static" vehicles.

    - **Time:** 3,600s.
    - **Topic Metrics: Average Journey Time (AJT)** and **Global Network Throughput**. Compare the "Route Convergence" time (how fast the system adapts) after a simulated accident blocks a major arterial.

5. Urban Traffic "Digital Twin" & Forensic Replay

- **The Goal:** Achieve high-fidelity state reconstruction for accident forensics and traffic auditing.
- **Detailed Tasks:**
    - **Telemetry Logging:** Design a high-throughput schema to log (ID, x, y, speed, angle) for every vehicle at every step.
    - **Batch Ingestion:** Optimize the Python-to-SQLite pipeline using executemany for real-time logging.
    - **Conflict Detection:** Use SQL triggers to identify "Near-Misses" (vehicles within d < 2m while speed > 0).
    - **State Replay:** Write a script that reads a specific timestamp from the DB and recreates that exact vehicle distribution in a new SUMO instance.
    - **Trend Analysis:** Aggregate telemetry to identify roads with the highest "sudden braking" frequency.
- **Performance Evaluation Scale:**
    - **Network:** 2km × 2km high-density downtown.
    - **Scale:** 1,500 vehicles.
    - **Time:** 1,800s (30-minute high-resolution window).
    - **Topic Metrics: Forensic Reconstruction Accuracy** (Mean Squared Error of x,y coordinates) and **Incident Detection Recall**. Measure the storage footprint vs. the ability to reconstruct a collision within 50cm of accuracy.

6. Adaptive Traffic Signal Control (ATSC) via Relational Logic

- **The Goal:** Reduce intersection delay by treating traffic lights as DB-driven actuators.
- **Detailed Tasks:**
    - **Infrastructure Mapping:** Store all traffic light phases and induction loop associations in the DB.
    - **Sensor Streaming:** Stream induction loop counts into an SQLite Sensor_Data table.
    - **Pressure Logic:** Implement a "Max-Pressure" controller via SQL queries that determine the phase with the highest waiting volume.
    - **Signal Actuation:** Update SUMO traffic light states via TraCI based on the DB's decision.
    - **Adaptive Feedback:** Use historical throughput data in the DB to adjust the "Minimum Green Time" parameters.
- **Performance Evaluation Scale:**
    - **Network:** 10 interconnected signalized intersections.
    - **Scale:** 2,500 vehicles flowing through a corridor.
    - **Time:** 3,600s peak hour.
    - **Topic Metrics: Average Delay per Vehicle** and **Maximum Queue Length**. Evaluate the "Systemic Throughput" compared to a fixed-time Webster cycle

baseline.

7. Predictive Road Maintenance and Asset Management

- **The Goal:** Balance road longevity with traffic flow by simulating cumulative damage.
- **Detailed Tasks:**
    - **Durability Schema:** Create an Edge_Health table; roads lose "health" based on vehicle weight and volume.
    - **Cumulative Damage:** Every time a "Truck" class vehicle passes an edge in TraCI, execute a SQL UPDATE to decrement health.
    - **Failure Simulation:** When health drops to zero, use TraCI to close the lane or reduce the speed limit to 10km/h.
    - **Predictive Querying:** Use SQL to identify the "Top 5 edges likely to fail in the next 1000 steps."
    - **Maintenance Planning:** Design a SQL-based scheduler that prioritizes repairs on high-traffic roads.
- **Performance Evaluation Scale:**
    - **Network:** 50km highway network with 20 bridge/tunnel assets.
    - **Scale:** 10,000 total vehicle passes (simulated over accelerated multi-day cycles).
    - **Time:** 43,200s (12-hour accelerated window).
    - **Topic Metrics: Network Uptime (%)** and **Infrastructure Health Variance**. Analyze how the DB-driven maintenance priority affects the "Life-Cycle Cost" of the road segments.

Field 3: Logistics & Emerging Mobility

8. EV Fleet Charging Logistics and Grid Balancing

- **The Goal:** Manage electric vehicle fleets to ensure trip completion while avoiding charging station congestion.
- **Detailed Tasks:**
    - **Battery Tracking:** Track State-of-Charge (SoC) for all EVs in a relational table.
    - **Low-Battery Events:** Use a trigger/poller to detect EVs with SoC < 15%.
    - **Charging Queue:** Implement a SQL-based queue for charging stations, prioritizing vehicles with the lowest SoC.
    - **Rerouting:** Use TraCI to send EVs to the station assigned by the DB.
    - **Load Prediction:** Aggregate current EV destinations to predict which charging stations will be overloaded in the next hour.
- **Performance Evaluation Scale:**
    - **Network:** Urban area with 25 charging stations.
    - **Scale:** 600 EVs in a 3,000 vehicle total population.
    - **Time:** 43,200s (12-hour simulation cycle).
    - **Topic Metrics: Fleet Downtime (Charging Wait Time)** and **Grid Peak-to-Average Ratio**. Evaluate the "Range Anxiety" factor—the percentage of EVs that failed to reach a charger.

9. Multi-Modal "Mobility-as-a-Service" (MaaS) Integration

- **The Goal:** Optimize door-to-door travel times by synchronizing disparate transit modes.
- **Detailed Tasks:**
    - **MaaS Schema:** Combine bus schedules with real-time taxi availability in a single database.
    - **Intermodal Routing:** Write a SQL join that finds the fastest "Taxi-to-Bus" transfer point for a user.
    - **Dynamic Sync:** Use TraCI to track bus delays; the DB should automatically alert the taxi to "wait" or "speed up" to meet the bus.
    - **User Personalization:** Store user preferences (e.g., "Prefer Bus over Taxi") as weights in the matching query.
    - **Throughput Reporting:** Use SQL to calculate the average "Intermodal Transfer Time."
- **Performance Evaluation Scale:**
    - **Network:** 15 bus lines; 120 bus stops; 250 taxis.
    - **Scale:** 5,000 unique user journeys; 3,000 background cars.
    - **Time:** 7,200s.
    - **Topic Metrics: Average Intermodal Transfer Time** and **Total Journey Reliability (%)**. Compare wait times when buses "wait" for late-arriving taxi passengers via DB synchronization.

10. Connected Platoon Management via Recursive Hierarchies

- **The Goal:** Use recursive data structures to manage high-density vehicle strings for increased road capacity.
- **Detailed Tasks:**
    - **Hierarchy Schema:** Represent platoons using a self-referential table (Vehicle_ID, Leader_ID).
    - **Join/Leave Logic:** Use TraCI to detect proximity; execute a SQL transaction to "join" a follower to a leader.
    - **Recursive Update:** Use a **Recursive CTE** to propagate a speed change from the leader to all N followers in the DB.
    - **Dissolution Management:** When a leader exits the highway, use a SQL query to promote the second vehicle to leader and update the hierarchy.
    - **Efficiency Audit:** Log the "Headway" distance of platooned vs. non-platooned vehicles.
- **Performance Evaluation Scale:**
    - **Network:** 10km, 3-lane highway.
    - **Scale:** 500 heavy-duty trucks; 1,000 passenger cars.
    - **Time:** 3,600s.
    - **Topic Metrics: Lane Throughput (Veh/Lane/Hour)** and **Platoon Stability** (damping of shockwaves). Measure the "String Stability"—how far back a braking event propagates through the DB-managed hierarchy.

Field 4: Safety & Policy Enforcement

11. Emergency Service Vehicle (ESV) Corridor Preemption

- **The Goal:** Minimize response times for emergency vehicles by preemptively clearing traffic.
- **Detailed Tasks:**
    - **Priority Schema:** Maintain a table of active ESV dispatches and their current GPS coordinates.
    - **Preemption Zone:** Use an R-Tree query to identify all traffic lights and civilian vehicles 500m ahead of the ESV.
    - **Active Control:** Use TraCI to force civilian vehicles to the outer lanes and turn traffic lights green for the ESV corridor.
    - **Conflict Resolution:** Use the DB to manage two ESVs approaching the same intersection from different directions.
    - **Impact Analysis:** Log the delays caused to civilian traffic during the preemption.
- **Performance Evaluation Scale:**
    - **Network:** 4km × 4km urban center.
    - **Scale:** 20 active ESVs; 2,500 civilian vehicles.
    - **Time:** 3,600s (randomized incident dispatches).
    - **Topic Metrics: ESV Mean Response Time** and **Civilian Delay Penalty (seconds/vehicle)**. Plot the response time reduction vs. the "Congestion Ripple Effect" distance in the DB.

12. Privacy-Preserving Trajectory Masking & Data Utility

- **The Goal:** Provide useful traffic analytics while ensuring individual vehicle privacy.
- **Detailed Tasks:**
    - **Dual Schema:** Store Raw_Data (internal) and Anonymized_Data (public view).
    - **Noise Injection:** Apply Laplacian noise to TraCI coordinates before committing to the public DB.
    - **K-Anonymity:** Implement SQL views that only show traffic counts if at least K vehicles are in a zone.
    - **Utility Testing:** Run a traffic density query on both tables and compare results.
    - **Privacy Audit:** Attempt to "re-identify" a specific vehicle ID based on movement patterns in the public DB.
- **Performance Evaluation Scale:**
    - **Network:** 3 km × 3 km arterial grid.
    - **Scale:** 4,000 moving vehicles.
    - **Time:** 3,600s.
    - **Topic Metrics: Query Utility (RMSE of Density Maps)** and **Privacy Disclosure Probability**. Measure the "Noise vs. Accuracy" trade-off for traffic planners.

13. Smart Parking Allocation & Cruising Reduction

- **The Goal:** Eliminate "cruising for parking" through a real-time reservation database.
- **Detailed Tasks:**
    - **Parking Schema:** Map SUMO parking areas to a Parking_Spots table.
    - **Cruising Detection:** Identify vehicles circling a destination via TraCI and log their status.
    - **Reservation Engine:** When a vehicle enters the city, use SQL to assign and reserve the nearest spot.
    - **Price Response:** Implement dynamic pricing in the DB; if occupancy is > 90%, prices increase, and vehicles may choose to park further away.
    - **Inventory Audit:** Log total search time and fuel wasted by vehicles without a reservation.
- **Performance Evaluation Scale:**
    - **Network:** 800 on-street parking spots; 50 off-street lots.
    - **Scale:** 2,500 vehicles with destinations in the CBD.
    - **Time:** 7,200s.
    - **Topic Metrics: Average Search Time for Parking** and **Cruising-related VMT**. Evaluate the "System-wide Fuel Savings" achieved by eliminating circular cruising.

14. Road Safety Auditing via Near-Miss Self-Joins

- **The Goal:** Use the DB to proactively identify dangerous road designs before accidents occur.
- **Detailed Tasks:**
    - **Proximity Logging:** Log every vehicle's position at every step.
    - **Self-Join Query:** Run a query that finds all pairs of vehicles (V_1, V_2) where distance d < 1m and speed > 0.
    - **Hotspot Mapping:** Use SQL to count near-misses per intersection.
    - **Temporal Correlation:** Correlate near-miss frequency with specific traffic signal phases stored in the DB.
    - **Mitigation Test:** Modify the SUMO network (e.g., add a stop sign) and re-run the audit.
- **Performance Evaluation Scale:**
    - **Network:** 5 major intersections with varying geometries.
    - **Scale:** 3,000 vehicles.
    - **Time:** 3,600s high-density simulation.
    - **Topic Metrics: Conflict Frequency Index (CFI)** and **Time-to-Collision (TTC) Distribution**. Identify if a specific signal phase in the DB correlates with a 50% spike in near-misses.

15. Distance-Based Dynamic Road Tolling

- **The Goal:** Reduce congestion in city centers using transparent, DB-driven financial transactions.
- **Detailed Tasks:**
    - **Wallet Schema:** Manage user balances and vehicle entry/exit logs.
    - **Dynamic Pricing:** Calculate tolls based on real-time congestion levels (count of vehicles in the zone).

    - **Transaction Processing:** Use ACID transactions to deduct funds and log the movement.
    - **Diversion Analysis:** Track if vehicles are taking longer, non-tolled routes to avoid fees.
    - **Revenue View:** Generate an hourly report of city revenue vs. average vehicle speed.
- **Performance Evaluation Scale:**
    - **Network:** 2km × 2km tolled CBD zone.
    - **Scale:** 4,000 vehicles entering/exiting.
    - **Time:** 10,800s (3-hour morning peak).
    - **Topic Metrics: Peak Speed Increase** and **Zone Entry Elasticity**. Evaluate the "Revenue-to-Delay" ratio—how much delay is reduced per dollar of toll collected.

Field 5: Logistics & Network Resilience

16. Last-Mile Logistics: Truck-Drone Rendezvous Coordination

- **The Goal:** Maximize delivery efficiency by coordinating trucks and drones in a shared spatial DB.
- **Detailed Tasks:**
    - **Agent Schema:** Track locations and battery levels for Trucks and Drones.
    - **Rendezvous Logic:** Use SQL to find the best SUMO edge for a truck to meet a drone for battery swaps/reloads.
    - **Handover Logic:** Use transactions to transfer "Parcel" ownership from Truck to Drone.
    - **Path Sync:** Use TraCI to hold the truck if the drone is delayed (queried from DB).
    - **Efficiency Audit:** Compare "Parcels Delivered per Hour" against a truck-only baseline.
- **Performance Evaluation Scale:**
    - **Network:** 2km × 2km residential area.
    - **Scale:** 30 trucks; 150 drones; 1,000 parcels.
    - **Time:** 14,400s (4-hour window).
    - **Topic Metrics: Delivery Throughput (Parcels/Hour)** and **Energy Cost per Delivery**. Measure the "Idle Time" for trucks waiting on drones via DB-status logs.

17. Micromobility Rebalancing via Predictive Demand

- **The Goal:** Maintain e-scooter availability using historical demand patterns in SQL.
- **Detailed Tasks:**
    - **Scooter Schema:** Track ID, position, and status (available/in-use).
    - **Demand Mapping:** Log all "User Requests" and successful rentals.
    - **Imbalance Detection:** Use SQL to find zones where demand > supply.
    - **Rebalancing Task:** Dispatch a "Rebalancing Truck" in SUMO to move units from low-demand to high-demand areas.
    - **Availability Audit:** Calculate the "Service Level" (% of users finding a scooter

within 100m).

- **Performance Evaluation Scale:**
    - **Network:** 3 km × 3 km university/business district.
    - **Scale:** 500 e-scooters; 80 docking stations; 2,000 users.
    - **Time:** 86,400s (Full 24-hour cycle).
    - **Topic Metrics: Service Availability Rate (%)** and **Rebalancing Truck Odometer**. Evaluate the "Service Level" improvement after applying the predictive SQL model.

18. Dynamic Bus Lane (DBL) Activation Policy

- **The Goal:** Improve transit throughput by converting lanes to "Bus-Only" based on real-time DB triggers.
- **Detailed Tasks:**
    - **Lane Schema:** Store the state of every lane (General/Bus-Only).
    - **Delay Monitor:** Use TraCI to monitor bus delays; if delay > 5 mins, trigger a lane change in the DB.
    - **Infrastructure Update:** Use TraCI's setLaneAllowed to physically change the lane access in SUMO.
    - **Fairness Audit:** Use SQL to compare the delay of bus passengers versus car drivers.
    - **Automatic Reversion:** When bus volume drops, the DB trigger reverts the lane to general use.
- **Performance Evaluation Scale:**
    - **Network:** 6km arterial corridor with 4 lanes.
    - **Scale:** 25 buses/hour; 4,000 passenger cars/hour.
    - **Time:** 7,200s (Peak window).
    - **Topic Metrics: Person-Throughput (Pax/Hour)** and **Transit Delay Reduction (%)**. Measure the "Systemic Benefit"—did the bus time savings outweigh the car delay increase?

19. Recursive Graph Resilience under Major Disruptions

- **The Goal:** Maintain connectivity in a failing network using recursive graph audits.
- **Detailed Tasks:**
    - **Graph Adjacency Schema:** Represent the road network as a graph.
    - **Failure Injection:** Use TraCI to simulate multiple road closures (e.g., flooding).
    - **Reachability Analysis:** Use a **Recursive CTE** to find all nodes still reachable from the city center.
    - **Dynamic Detouring:** For isolated vehicles, query the DB for the "Last Possible Path" before the network fragments.
    - **Robustness Score:** Calculate the percentage of nodes connected over time.
- **Performance Evaluation Scale:**
    - **Network:** City-scale network (2,500 nodes; 6,000 edges).
    - **Scale:** 5,000 vehicles.
    - **Time:** 10,800s disaster scenario.
    - **Topic Metrics: Network Connectivity Index** and **Average Detour Length**

Increase. Analyze the "Threshold of Fragmentation"—at what point do DB queries fail to find any valid path?

20. Public Transit "Connectivity Guarantee" Sync

- **The Goal:** Ensure passengers never miss the "last bus" due to minor traffic delays.
- **Detailed Tasks:**
  - **Schedule Schema:** Store static schedules and real-time feeder shuttle locations.
  - **Sync Query:** Find all passengers on a "Feeder" vehicle who have a "Connecting" bus trip.
  - **Hold Logic:** If the feeder is late, the DB sends a "Hold" command to the connecting bus via TraCI.
  - **Delay Propagation:** Use SQL to calculate if holding the bus will cause it to miss *its* next connection.
  - **Success Logging:** Track the number of "Saved Connections."
- **Performance Evaluation Scale:**
  - **Network:** 20 bus lines; 10 tram lines; 150 stops.
  - **Scale:** 3,000 transfer-dependent passengers.
  - **Time:** 14,400s (4-hour simulation).
  - **Topic Metrics: Connection Reliability (%)** and **System-Wide Passenger Delay**. Evaluate the reduction in "Total Travel Time" for transfer passengers.

---

21. Real-Time "Profit-Aware" Routing for Commercial Delivery

- **The Goal: Optimize routing not just for time, but for the "Net Profit" of a delivery leg by factoring in fuel costs, toll fees, and time-penalties stored in a DB.**
- **Detailed Research Tasks:**
  - **Cost Schema: Create a Route_Economics table mapping edge IDs to dynamic fuel consumption estimates and time-of-day tolling.**
  - **Weight Injection: Implement a Python-based Dijkstra algorithm that queries the DB for "Profit-Weights" rather than standard travel times.**
  - **Delivery Window Constraint: Use SQL to filter potential routes that violate the "Must-Deliver-By" timestamp for a specific order.**
  - **TraCI Rerouting: Monitor real-time traffic jams; if the "Profit-Margin" of the current route drops below a threshold, trigger a re-calculation.**
  - **Historical Audit: Log the final profit of every completed trip to analyze the accuracy of the DB-based cost predictions.**
- **Performance Evaluation Scale:**
  - **Network: 5 km * 5 km urban network with 10 tolled arterials.**
  - **Scale: 100 heavy delivery trucks; 3,000 passenger vehicles.**
  - **Time: 7,200s (Peak window).**
  - **Topic Metrics: Gross Profit per Kilometer and On-Time Delivery Rate (%).**

22. "Pressure-Based" Network Signal Coordination via DB-Exchange

- **The Goal: Implement a decentralized Max-Pressure control where intersections "Trade" queue-state information via a shared local database.**
- **Detailed Research Tasks:**
  - **Intersection State Schema: Store the current "Inflow" and "Outflow" counts for 10 adjacent intersections in the DB.**
  - **Pressure Calculation: Use a SQL view to calculate the "Weight" of each signal phase (Inflow minus Downstream Outflow).**
  - **Local Decision Logic: Write a Python controller for each intersection that queries its own and neighbors' pressure from the DB.**
  - **TraCI Signal Actuation: Switch the green-phase to the direction with the highest relative pressure.**
  - **Gridlock Audit: Log instances of "Spillback" (queues reaching the upstream intersection) into a Network_Failure table.**
- **Performance Evaluation Scale:**
  - **Network: 4 * 4 Manhattan-style grid (16 intersections).**
  - **Scale: 5,000 vehicles flowing through the grid.**
  - **Time: 7,200 s.**
  - **Topic Metrics: Network-Wide Average Speed and Standard Deviation of Queue Lengths.**

23. Proactive Deadlock Prevention in Shared-Ride Reservations

- **The Goal: Use SQL transaction isolation levels to prevent "Circular Dependency" deadlocks when two vehicles attempt to claim the same high-demand shared-ride leg.**
- **Detailed Research Tasks:**
  - **Reservation Schema: Design a Ride_Legs table where vehicle IDs "Lock" segments of a path for shared passengers.**
  - **Deadlock Detection Query: Write a SQL query to identify if a new reservation will create a "No-Win" reroute for existing passengers.**
  - **Transactional Rollback: Use SQLite BEGIN IMMEDIATE transactions to ensure that if a ride-share match fails the "Time-Window" test, the DB state reverts.**
  - **TraCI Sync: Only update the physical route in SUMO once the DB transaction is successfully committed.**
  - **Concurrency Audit: Log the number of "Transaction Retries" during high-demand bursts to evaluate system stability.**
- **Performance Evaluation Scale:**
  - **Network: 2 km * 2 km high-density CBD.**
  - **Scale: 400 shared taxis; 2,000 overlapping trip requests.**
  - **Time: 3,600 s (Massive demand spike).**
  - **Topic Metrics: System Throughput (Requests/Sec) and Transaction Success Latency.**

24. "Return-to-Home" Driver-Centric Ride-Sharing

- **The Goal: Increase driver retention by using the DB to prioritize shared-ride requests that end near the driver's own home/destination.**
- **Detailed Research Tasks:**

    - **Driver Preference Schema:** Store every driver's "Home Location" and "Desired End-Time" in the DB.
    - **Directional Matching:** Write a SQL query that filters passenger requests based on the "Angular Deviation" from the driver's home path.
    - **Utility-Based Dispatching:** Calculate a "Match-Score" that balances passenger wait-time with driver home-proximity.
    - **TraCI Routing:** Inject the final "Going-Home" route into the vehicle as it nears the end of its shift.
    - **Fairness Audit:** Use SQL to calculate the "Home-Arrival Delay" for all drivers in the fleet.
- **Performance Evaluation Scale:**
    - **Network: 10km * 10 km metropolitan area.**
    - **Scale: 300 drivers (cars); 1,200 passenger requests.**
    - **Time: 10,800 s (Evening shift end).**
    - **Topic Metrics: Mean Home-Arrival Error (km/minutes) and Fleet Dispatch Efficiency.**

25. Dynamic Pricing for Ride-Sharing based on Signal Delay

- **The Goal: Use the DB to link the cost of a ride-share trip to the real-time "Signal Delay" predicted on its route.**
- **Detailed Research Tasks:**
    - **Integrated Schema: Combine Signal_States and Ride_Quotes into a unified relational model.**
    - **Delay Prediction: Write a SQL query that estimates the "Signal Wait" for a specific route based on current DB phase cycles.**
    - **Surge-Logic: Increase the ride-share price in the DB if the route is experiencing high signal-induced congestion.**
    - **TraCI Rerouting: Offer the passenger a "Cheaper, Longer" route that avoids the signal-clogged arterials.**
    - **Revenue vs. Throughput Audit: Analyze if price-driven rerouting improves overall intersection throughput.**
- **Performance Evaluation Scale:**
    - **Network: 3 km * 3 km grid with 9 intersections.**
    - **Scale: 500 ride-share vehicles; 1,500 price-sensitive users.**
    - **Time: 7,200 s.**
    - **Topic Metrics: Average Trip Fare and Intersection V/C (Volume-to-Capacity) Ratio.**